

MANUAL26. УРОК 16. МОБИЛЬНОЕ ТЕСТИРОВАНИЕ: КАК ПРОВЕРЯТЬ ПРИЛОЖЕНИЕ В УСЛОВИЯХ РЕАЛЬНОГО ПОЛЬЗОВАТЕЛЯ.

Почему мобильная проверка почти всегда сложнее desktop?

Потому что мобильный пользователь постоянно меняет контекст: переключается между сетями, получает звонки, сворачивает приложение, возвращается через минуту, меняет ориентацию экрана, работает при низком заряде. На desktop таких резких переходов меньше, а в мобайле они — норма.

Как этот урок связан с кросс-платформенным покрытием?

В уроке 15 вы научились выбирать правильные среды. Теперь мы углубляем самую сложную из них — мобильную. Здесь важно не только «работает/не работает», но и устойчивость сценария после смены условий.

Что в мобильном контексте считается реальным качеством?

Качество — это когда пользователь может завершить цель даже после прерываний, сетевых сбоев и фоновых переходов. Если сценарий ломается при обычном звонке или коротком обрыве сети, такой дефект для мобильной аудитории может быть критичнее, чем кажется на стенде.

Какие риски чаще всего недооценивают

начинающие?

Обычно недооценивают:

1. восстановление после потери сети;
2. поведение после возврата из фона;
3. рассинхронизацию состояния клиента и сервера;
4. влияние слабых устройств на UX;
5. накопительные эффекты долгой сессии.

Эти риски редко видны в «чистом» сценарии, но именно они чаще доходят до пользователей.

Какие группы проверок стоит держать обязательными?

Минимальный каркас мобильной проверки:

1. функциональный сценарий от старта до завершения;
2. ввод и жесты;
3. сеть и восстановление;
4. поворот экрана и адаптация UI;
5. прерывания и фон;
6. производительность базового уровня.

*Если хотя бы один блок
игнорируется, картина качества
получается неполной.*

Почему сеть — главный источник мобильных дефектов?

Потому что проблема часто
проявляется не в момент
«offline», а при возврате связи.
Приходят запоздалые ответы,
перезаписываются новые
данные, интерфейс зависает в
загрузке, пользователь не
понимает, что делать дальше.
Именно этап восстановления
ломает реальный опыт.

Что важно проверять в сетевых переходах?

Не только «есть интернет / нет
интернета», а:

1. сохраняются ли данные
пользователя;
2. не дублируются ли запросы и
действия;
3. получает ли пользователь
ясный статус;
4. может ли продолжить
сценарий без перезапуска;

5. корректно ли работает повторная попытка.
-

Почему прерывания критичны для мобильного UX?

Звонки, уведомления и блокировка экрана происходят постоянно. Если после возврата кнопки неактивны, данные сброшены или интерфейс «рассыпался», пользователь считает приложение нестабильным даже при идеальной основной логике.

Как тестировать прерывания системно, а не случайно?

Полезно запускать сценарий по шагам:

1. дойти до ключевой точки (например, оформление покупки);
2. инициировать прерывание;
3. вернуться и проверить состояние;
4. завершить сценарий;
5. убедиться, что результат корректно сохранен.

*Так вы проверяете не фрагмент,
а целостность пути.*

Как проверять фоновые переходы без лишней «магии»?

Переходите приложение в фон
на разные интервалы (короткий/
средний/длинный),
возвращайтесь и фиксируйте:

1. сохранился ли контекст
экрана;
 2. активны ли СТА-элементы;
 3. не появился ли повторный
запрос;
 4. не устарели ли данные;
 5. есть ли необходимость
ручного рефреша.
-

Почему поворот экрана все еще источник багов?

Потому что в повороте
одновременно меняются layout,
размеры элементов, фокус ввода
и иногда состояние экрана. Если
приложение не обрабатывает это
аккуратно, появляются
наложения, сдвиги, «мертвые
зоны» тапа и потеря введенных
значений.

Как выбирать устройства для мобильной проверки?

Покрывать все модели невозможно, поэтому применяют риск-ориентированный выбор:

1. массовые устройства аудитории;
2. несколько слабых устройств;
3. актуальные версии ОС;
4. исторически проблемные сочетания;
5. хотя бы один «эталон» для сравнения.

Важно: выбор должен быть обоснован данными, а не только доступностью девайса на столе.

Как это выглядит на примере Brawl Stars?

Добавлен новый экран сезонных заданий. В идеальной сессии все проходит. После поворота часть карточек наслаивается, после звонка кнопка входа в бой становится неактивной, после краткого обрыва сети прогресс обновляется с задержкой. Это типичный мобильный кейс: логика вроде работает, но опыт пользователя ломается.

Что фиксировать в баг-репорте по мобильному дефекту обязательно?

Минимальный набор:

1. модель устройства;
2. версия ОС;
3. сборка приложения;
4. тип сети и условия;
5. шаги и частота;
6. поведение до/после прерывания;
7. артефакты (видео/скрин).

Без этих данных дефект сложнее воспроизвести и оценить.

Какие ошибки чаще всего делают начинающие QA?

Самые частые:

1. тестируют только на стабильном Wi-Fi;
2. не проверяют фоновые переходы;
3. пропускают прерывания в сценариях;
4. проверяют один класс устройств;
5. не фиксируют окружение в баге;

6. смешивают функциональную ошибку и UX-деградацию.

Как быстро понять, что мобильная сессия закрыта качественно?

Быстрый чек:

1. ключевые сценарии прошли с реальными прерываниями;
 2. проверены сетевые переходы и восстановление;
 3. зафиксированы данные окружения;
 4. описан риск для пользователя;
 5. понятен список сценариев для постоянного мобильного регресса.
-

Почему мобильное тестирование — это про устойчивость, а не только про функциональность?

Потому что пользователь оценивает приложение не в лабораторных условиях, а в «шумной» реальности. Если продукт выдерживает эту

реальность, значит качество действительно высокое.

Какой главный вывод урока?

Сильный мобильный QA проверяет не только «что работает», но и «что происходит после сбоя контекста». Именно это снижает долю неприятных сюрпризов после релиза.

Какой мостик к следующему уроку?

После мобильных проверок важно правильно донести результат до команды. В уроке 17 разберем отчетность QA: как превращать наблюдения в понятное релизное решение.

Почему на мобильных проектах важны длинные сессии тестирования?

Потому что часть дефектов проявляется не в первые минуты, а после накопления

состояния: кэш, повторные переходы, нагрев устройства, фоновая активность. Если проверять только короткие сценарии, можно пропустить деградацию, которая бьет по активным пользователям.

Как учитывать влияние батареи и производительности устройства?

Низкий заряд и режим энергосбережения могут менять поведение системы: ограничивать фоновые задачи, снижать частоту обновления, агрессивнее выгружать приложение из памяти. Эти условия стоит отдельно прогонять для критичных сценариев входа, покупки и возврата.

Почему важно проверять сценарии на разных типах сети, а не только Wi-Fi?

Потому что пользователи регулярно переходят между Wi-Fi и мобильной сетью. В этих переходах часто ломаются

таймауты, повторные запросы и отображение статусов. Проверка только на стабильном Wi-Fi создает ложное чувство надежности.

Как выявлять «плавающие» мобильные баги?

Полезный подход — серия повторов в одинаковых условиях с записью частоты. Например: 10 повторов сценария «фон → возврат → действие». Даже если ошибка проявляется 2–3 раза, это уже сигнал к риску и отдельной эскалации.

Что делать, если дефект воспроизводится только на одном устройстве?

Сначала подтвердить, не является ли это общим классом проблемы: проверить на устройстве той же линейки/ОС или схожей производительности. Если повторяется, это уже не «единичный случай», а риск для сегмента пользователей.

Какой дополнительный практический эффект дает урок?

Вы начинаете мыслить
мобильное качество через
устойчивость к изменению
контекста. Это помогает
находить проблемы до релиза, а
не разбирать «непонятные»
продовые жалобы постфактум.

Почему стоит отдельно проверять сценарии после обновления приложения?

После апдейта пользователь
может вернуться в старую
сессию, иметь кэш от прошлой
версии и нестабильную сеть в
момент первого запуска. Эти
условия часто дают дефекты
миграции состояния, которые не
видны в «чистой установке».

Как учитывать push- уведомления в мобильных сценариях?

Push может вмешиваться в критичный поток в любой момент: во время оплаты, подтверждения действия, заполнения формы. Полезно проверять, не ломают ли уведомления фокус экрана и сохранность данных после возврата.

Почему стоит отдельно смотреть работу приложения при нехватке памяти?

На слабых устройствах система агрессивно выгружает процессы. Если приложение не умеет корректно восстановить контекст после выгрузки, пользователь теряет прогресс и доверие к продукту. Это один из частых источников «плавающих» жалоб.

Как оценивать мобильный дефект с точки зрения бизнеса?

Через влияние на ключевые мобильные цели:

1. вход и удержание сессии;

2. завершение основного действия;
3. покупка/монетизация;
4. возврат пользователя после прерывания.

Если баг ломает эти точки, его приоритет часто выше, чем кажется по чисто техническому описанию.

Какие признаки показывают зрелый мобильный набор в команде?

Если команда регулярно проверяет сеть, фон, прерывания и классы устройств в каждом релизном цикле, а результаты фиксируются в едином формате, значит мобильная часть процесса стала системной, а не случайной.

Как использовать обратную связь пользователей для мобильного набора?

Жалобы из поддержки и сторов часто указывают на реальные сценарии, которых нет в лабораторной проверке. Если

включать эти паттерны в будущие тестовые циклы, набор быстрее адаптируется к реальному поведению аудитории.

Почему важно разделять функциональную стабильность и UX-устойчивость?

Сценарий может завершаться успешно, но сопровождаться задержками, «фризами» или неочевидными статусами после прерываний. С точки зрения пользователя это тоже дефект качества, поэтому QA должен фиксировать оба аспекта отдельно.

Что полезно повторить перед мобильной практикой?

Перед началом прогона проверьте, что в плане есть минимум один сценарий с сетевым переходом и один с прерыванием/возвратом из фона. Без этих условий мобильная

проверка редко отражает
реальный пользовательский
опыт.

Какой быстрый показатель качества мобильного цикла?

Если после каждого релиза у
команды уменьшается число
«неожиданных» дефектов,
связанных с фоном и сетью,
значит мобильный набор
действительно проверяет
жизненно важные условия, а не
только лабораторный сценарий.

Глоссарий к занятию

1. **Мобильное прерывание** —
внешнее событие, которое
меняет состояние приложения
(звонок, уведомление,
блокировка).
2. **Сетевой переход** — смена
типа соединения или
временная потеря сети во
время сценария.
3. **Фоновый режим** —
состояние, когда приложение
не активно на экране и
частично ограничено
системой.
4. **Устойчивость состояния** —
способность приложения

корректно продолжать работу
после изменения условий.

5. **Теплый старт** — возврат в приложение после короткого нахождения в фоне.